

1. Arbres binaires de recherche
2. Opérations de base
3. ADT TreeMap
4. Performances
5. Arbres de recherche équilibrés/balancés
6. Opérations de rotation et re-liaison (pour équilibrer)
7. Opération de restructuration de trois noeuds

Dans ce module, nous utilisons une structure arborescente de recherche pour implémenter efficacement l'ADT `SortedMap`.

L'ADT `SortedMap` inclut des fonctionnalités garantissant l'itération sur les clés dans l'ordre et prenant en charge des recherches supplémentaires, telles que `higherEntry( k )` et `subMap( k1, k2 )`.

Les arbres binaires sont une excellente structure de données pour stocker les entrées d'une `SortedMap`, en supposant que nous ayons une relation d'ordre définie sur les clés.

Rappel, les 3 opérations de gestion de l'ADT `Map` :

<code>get(k)</code>	Retourne la valeur v associée à la clé k , si elle existe, sinon retourne <code>null</code>
<code>put(k, v)</code>	S'il n'y a pas d'entrée avec la clé k , alors ajoute l'entrée $[k, v]$ et retourne <code>null</code> . Sinon, remplace avec v la valeur existante de l'entrée avec la clé k et retourne l'ancienne valeur
<code>remove(k)</code>	Retire l'entrée avec la clé k et retourne sa valeur. S'il n'y a pas d'entrée avec la clé k , retourne <code>null</code>

Voir le code de :

- `SortedMap.java` et
- `AbstractSortedMap.java`.

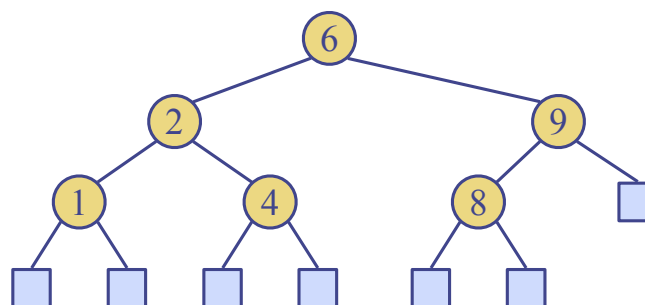
Rappel : Arbre Binaire de Recherche (ABR)

Dans le contexte de l'ADT `Map`, un arbre binaire de recherche est un arbre binaire stockant des entrées [clé-valeur] dans ses noeuds et satisfaisant la propriété suivante :

Soit u , v , et w , trois noeuds tels que u est dans le sous-arbre gauche de v et w est dans le sous-arbre droit de v . Nous avons que

$$\text{clé}[u] < \text{clé}[v] < \text{clé}[w]$$

- 👉 Dans un contexte plus général admettant des clés non uniques (`MultiMap`), nous avons que $\text{clé}[u] \leq \text{clé}[v] \leq \text{clé}[w]$ (**mais pas ici !!!**)
- 👉 Remarquez que dans un parcours dans l'ordre [inorder] d'un arbre binaire de recherche visite les clés en ordre croissant, dans l'exemple : 1, 2, 4, 6, 8, 9.



■ noeuds externes spéciaux null

Recherche dans un ABR

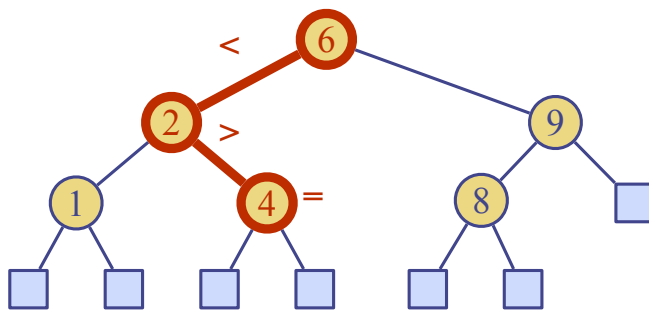
Pour **chercher** la clé k , on descend un chemin qui démarre à la racine

Le prochain noeud visité dépend de la comparaison de k avec la clé du noeud visité

Si nous atteignons `null`, la clé n'est pas trouvée

Exemple: **chercher** 4 dans l'ABR ci-dessous :

TreeSearch(T, T.racine[], 4)



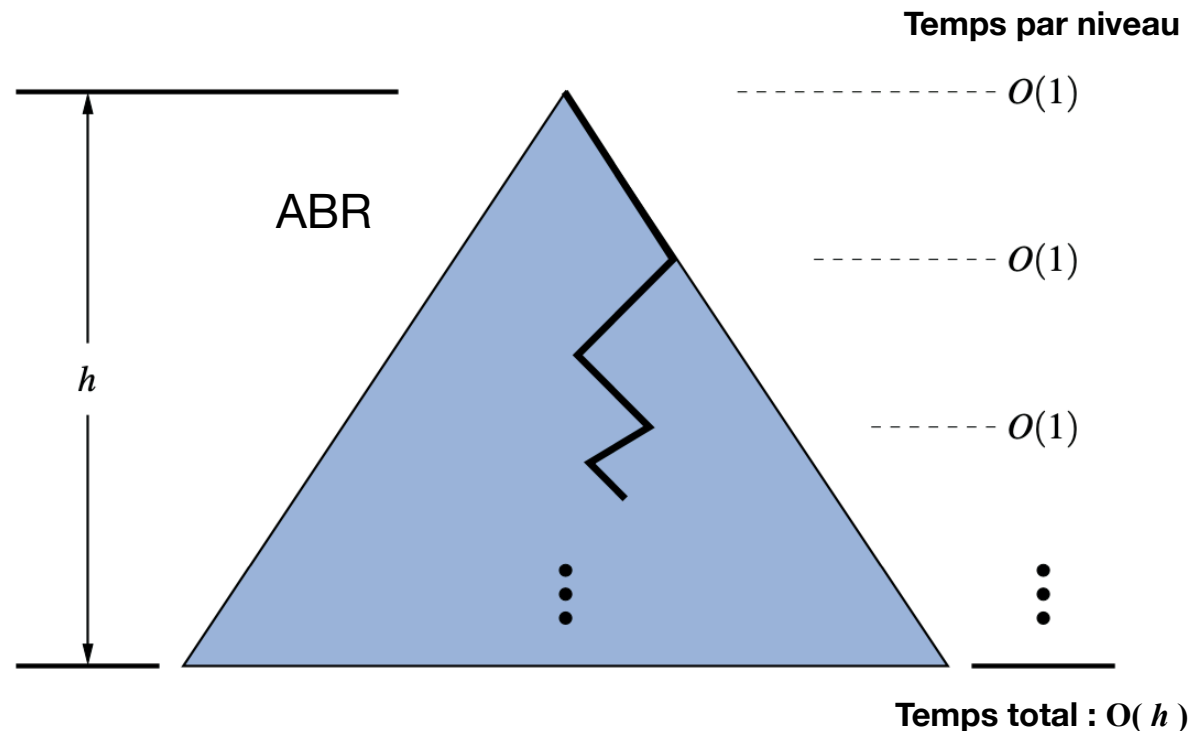
Algorithm TreeSearch(T, p, k):

```

if k == p.key() then
    return p                                {succès}
else if k < p.key() and T.left(p) is not None then
    return TreeSearch(T, T.left(p), k)       {appel récursif à gauche}
else if k > p.key() and T.right(p) is not None then
    return TreeSearch(T, T.right(p), k)      {appel récursif à droite}
return p                                    {échec}
  
```

Analyse de la recherche

L'algorithme `TreeSearch` est récursif et exécute un nombre constant d'opérations primitives pour chaque appel récursif. Chaque appel récursif est effectué sur un des enfants. C'est-à-dire que `TreeSearch` est appelé sur les noeuds d'un chemin qui commence à la racine et descend un niveau à la fois. Ainsi, le nombre de noeuds est limité par $h + 1$, où h est la hauteur de l'ABR. En d'autres termes, puisque nous passons $O(1)$ temps par noeud rencontré, la recherche globale se déroule en $O(h)$.



Dans le contexte de l'ADT `SortedMap`, la recherche est utilisée comme sous-routine pour implémenter les méthodes `get`, `put` et `remove`, puisque chacune commence par localiser une entrée existante avec la clé donnée. Toutes ces opérations s'exécuteront dans le pire des cas en $O(h)$ pour un arbre de hauteur h .

La hauteur h peut être aussi grande que le nombre d'entrées, n , (pire cas) mais nous nous attendons à ce qu'elle soit généralement beaucoup plus petite. Nous étudierons des stratégies pour maintenir h à une borne supérieure dans $O(\log n)$.

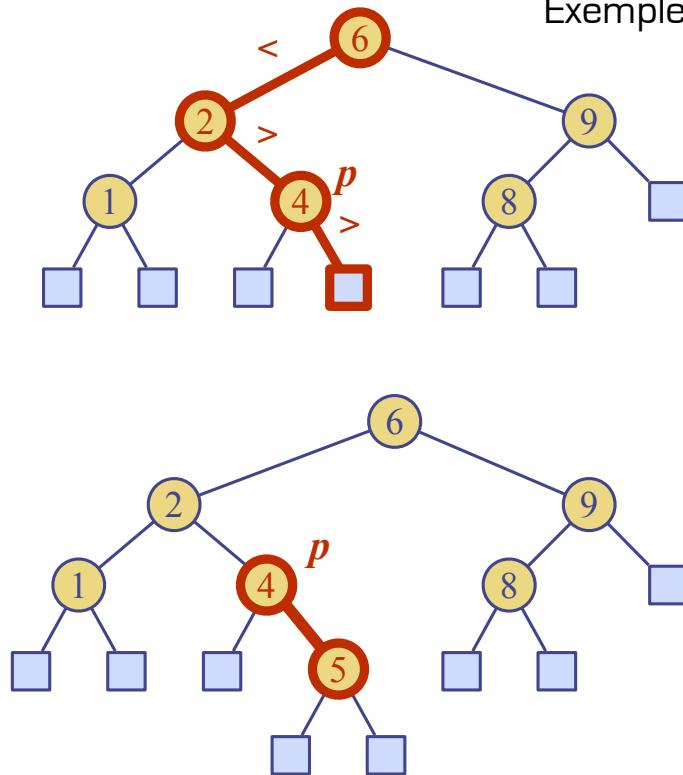
Insertion dans un ABR

Pour effectuer l'opération d'**insertion** [k, v], on cherche la clé k (en utilisant **TreeSearch**)

Supposons que k ne soit pas déjà dans l'arbre, et soit p le noeud où se termine la recherche

Nous insérons un nouveau noeud pour [k, v] comme enfant du noeud p

Exemple: **TreeInsert**($T, 5, v$)



Algorithm TreeInsert(T, k, v):

Input: A search key k to be associated with value v

$p = \text{TreeSearch}(T, T.\text{root}(), k)$

if $k == p.\text{key}()$ **then**

Set p 's value to v

else if $k < p.\text{key}()$ **then**

add node with item (k, v) as left child of p

else

add node with item (k, v) as right child of p

Suppression dans un ABR

Pour effectuer l'opération **supprimer**(k), on cherche d'abord la clé k

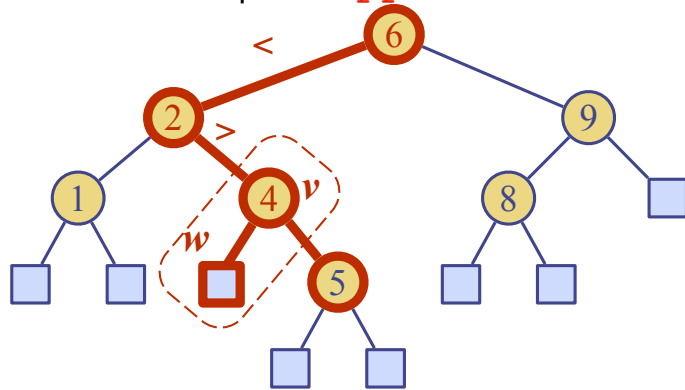
Supposons que la clé k est dans l'arbre, et que v soit le nœud stockant k

Si le nœud v possède au plus un enfant, nous le supprimons avec l'opération **remove**(v) de la classe `LinkedBinaryTree` - voir :

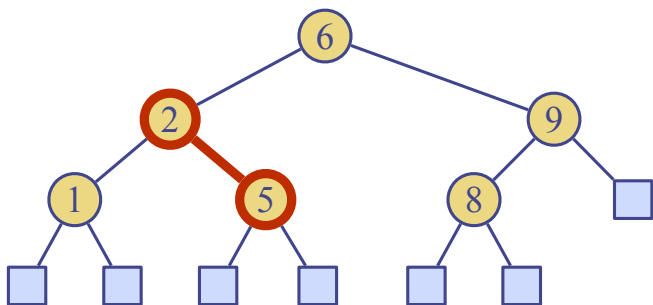
IFT2015/4. Arbres de recherche et tables/2. Arbres binaires.pdf

remove(v) supprime v et le remplace par son enfant s'il en a un

Exemple: **supprimer** 4



Voir le code de `remove` dans :
`LinkedBinaryTree.java`



Suppression dans un ABR [cas d'un noeud interne avec 2 enfants]

Nous considérons le cas où la clé k à supprimer est stockée sur un nœud v qui possède deux enfants

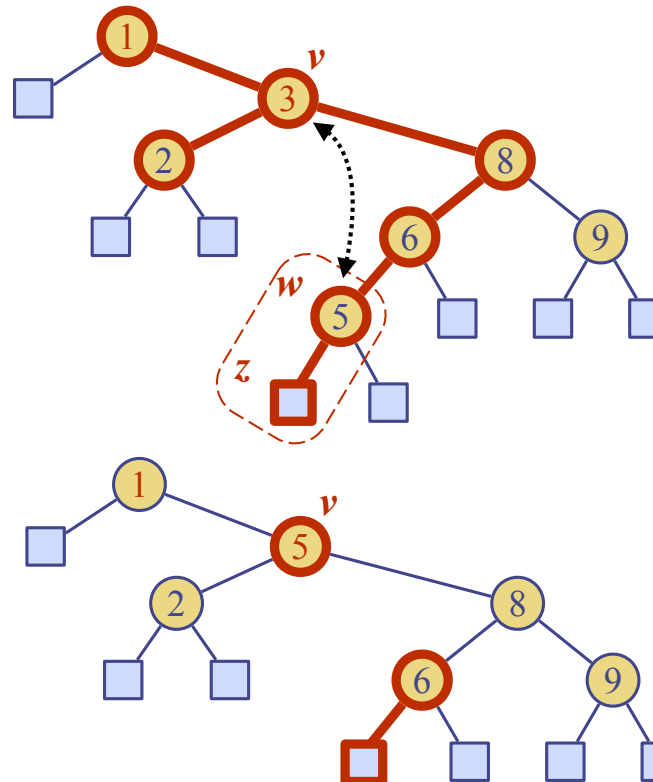
Nous trouvons le nœud, w , qui suit v dans un parcours en ordre [inorder]

☞ Alternativement, le nœud qui précède v dans un parcours inorder

Nous copions l'entrée [w , value] au nœud v

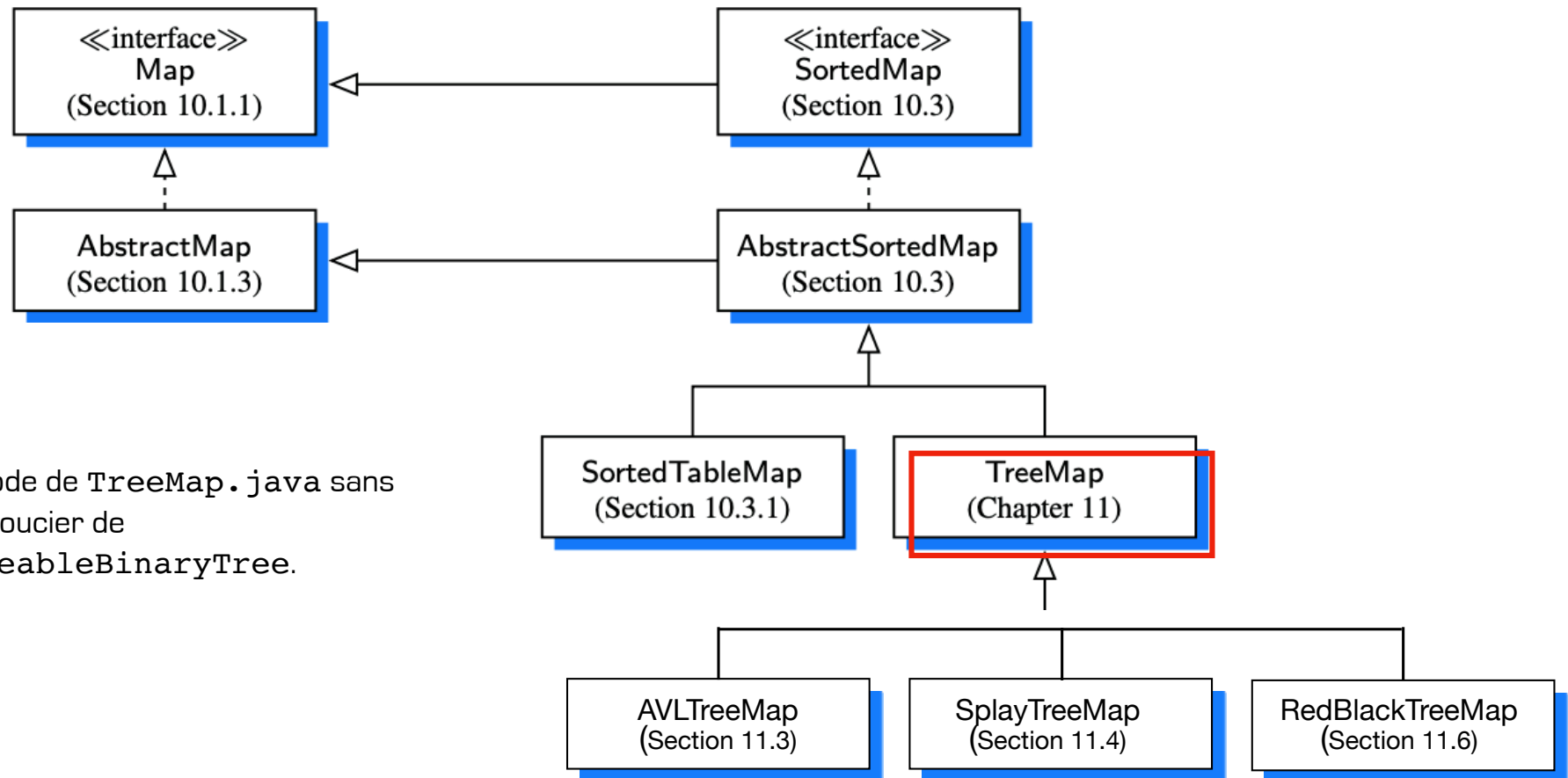
Nous enlevons le nœud w au moyen de l'opération `remove(  $w$  )` de `LinkedBinaryTree` qui supprime son enfant droit et fait monter z [enfant gauche]

Exemple: **supprimer** 3



☞ Soyez convaincu.e que le nœud w possède au plus 1 enfant !

TreeMap utilise un **ABR** pour implémenter SortedMap



Voir le code de `TreeMap.java` sans trop se soucier de `BalanceableBinaryTree`.

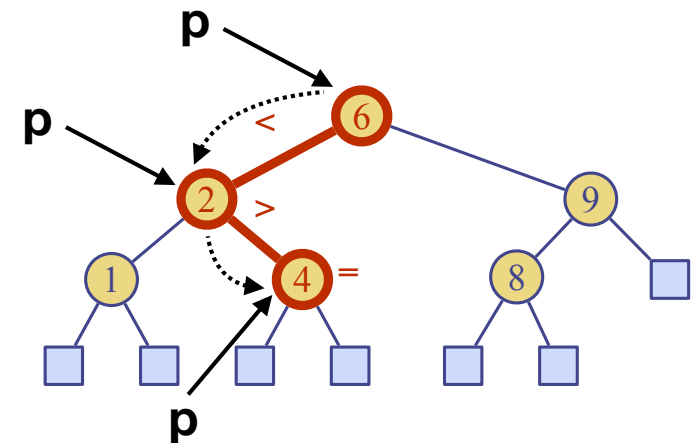
Les spécialisations de `TreeMap`, AVL, Splay et RedBlack, diffèrent de par leur manière de manipuler la structure de l'arbre, en particulier de le garder équilibré [on dit aussi balancé ; "balanced"].

Avant de regarder `BalanceableBinaryTree`, regardons les méthodes de `Map` et `SortedMap` dans le contexte d'un ABR.

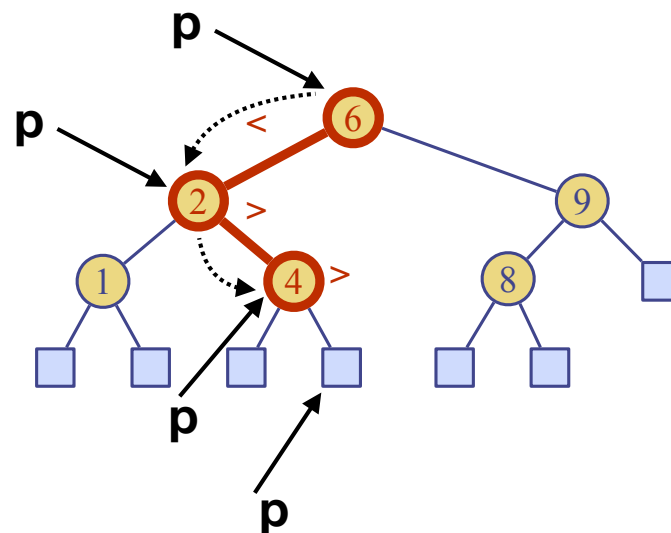
L'utilitaire `treeSearch` retourne la position d'un sous-arbre dont la racine possède la clé k , ou sinon une feuille sentinelle.

$\{k\}$: position du noeud dont la clé est k

```
treeSearch( {6}, 4 )
treeSearch( {2}, 4 )
treeSearch( {4}, 4 )
return {4}
```

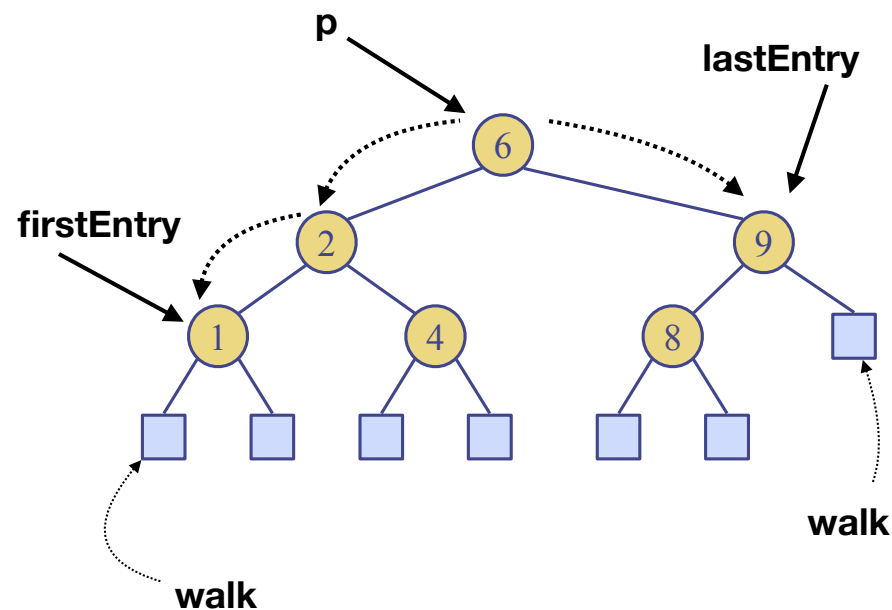


```
treeSearch( {6}, 5 )
treeSearch( {2}, 5 )
treeSearch( {4}, 5 )
treeSearch( {}, 5 )
return {enfant droit de {4}}
```



Comment gérer les intervalles ?

Voir les codes dans :
`TreeMap.java`



`treeMin` et `treeMax` sont 2 utilitaires qui descendent, respectivement à gauche et à droite, à partir d'une position, **p**. Ils retournent le parent de **walk**.

```
firstEntry = treeMin( root()).getElement()
```

```
lastEntry = treeMax( root()).getElement()
```

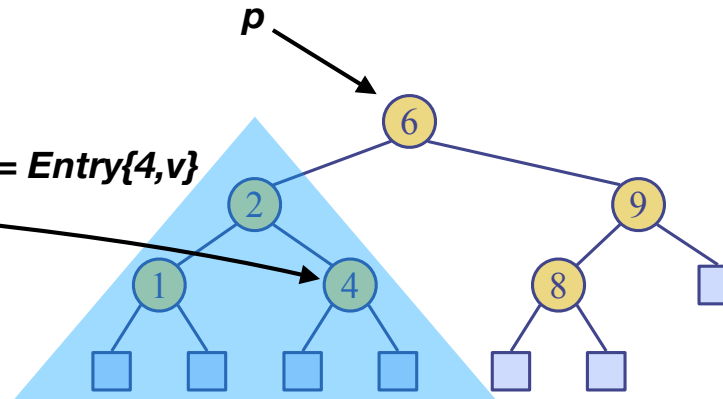
Voir les codes dans :
 TreeMap.java

{6} et **left{ {6} }** sont internes

$\text{left}(p) = \{2\}$

$\text{lowerEntry}(6) = \text{treeMax}(\text{left}(p)).\text{getElement}() = \text{Entry}\{4, v\}$

`lowerEntry` retourne
 l'entrée avec la clé la plus
 grande strictement inférieure
 à la clé donnée, si elle existe.

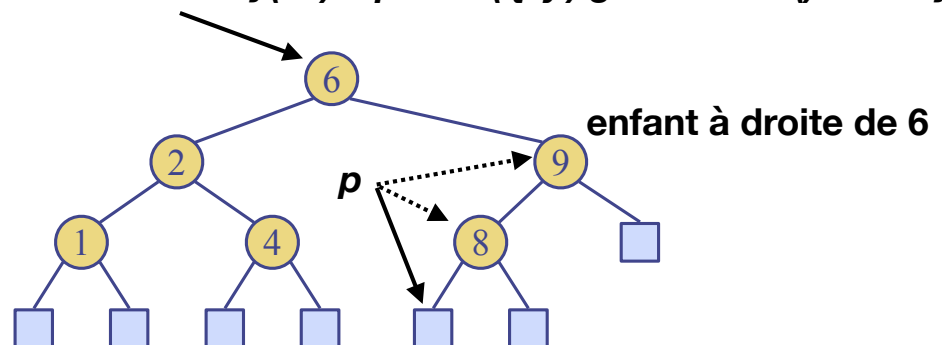


$\text{lowerEntry}(6)$ est l'entrée qui possède la plus grande clé du sous-arbre gauche...

noeud trouvé n'a pas d'enfant à gauche

`lowerEntry` retourne le
 premier ancêtre dont la clé
 est plus petite que celle du
 parent de **p** après une
 recherche infructueuse
 (lorsque la clé n'existe pas).

$\text{lowerEntry}(7) = \text{parent}(\{9\}).\text{getElement}() = \text{Entry}\{6, v\}$



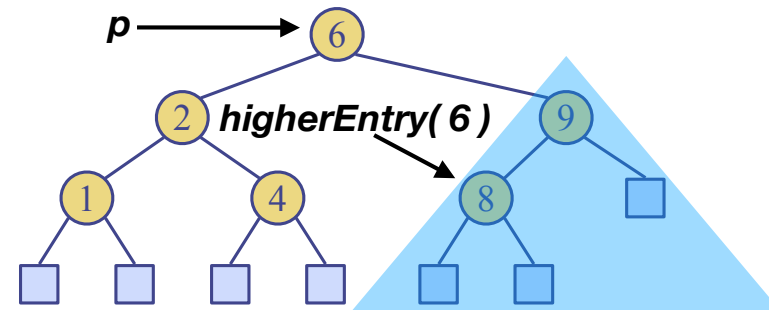
... ou le premier ancêtre dont la clé est plus petite que celle de **p**

👉 Lorsqu'on remonte, c'est le parent du premier noeud qui est un enfant de droite

Voir les codes dans :
 TreeMap.java

{6} et **right({6})** sont internes

higherEntry retourne
 l'entrée avec la clé la plus
 petite strictement
 supérieure à la clé donnée,
 si elle existe.



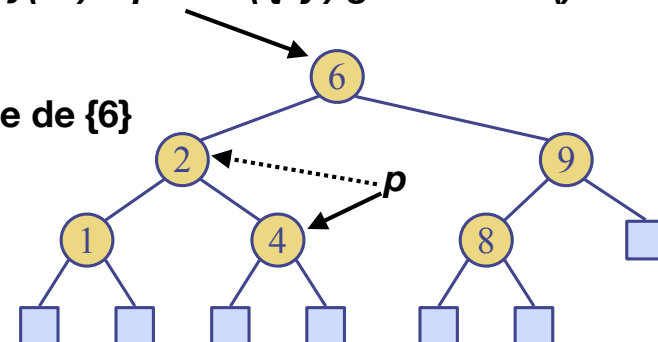
higherEntry est l'entrée qui possède la plus petite clé du sous-arbre droit...

{4} n'a pas d'enfant à droite

higherEntry retourne
 le premier ancêtre dont la
 clé est plus petite que celle
 du parent de **p** après une
 recherche infructueuse
 (lorsque la clé n'existe pas).

higherEntry(4) = parent({2}).getElement() = Entry{6,v}

{2} est l'enfant à gauche de **{6}**



... ou le premier ancêtre dont la clé est plus grande que celle de **p**

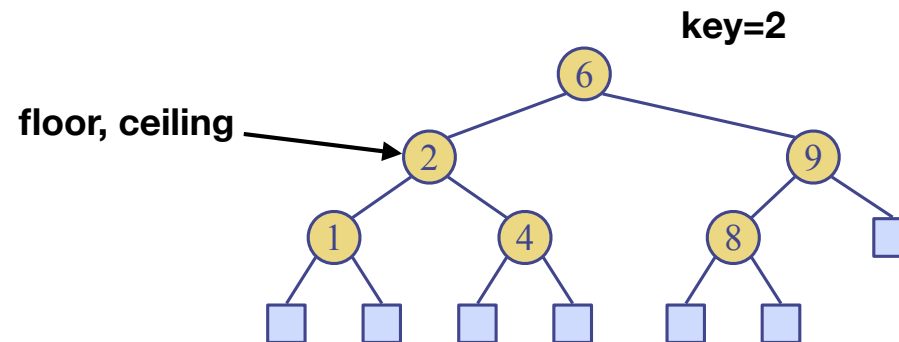
👉 Lorsqu'on remonte, c'est le parent du premier noeud qui est un enfant à gauche

Voir les codes dans :
`TreeMap.java`

`floorEntry` retourne
l'entrée avec la clé la plus
grande plus petite ou égale
à la clé donnée, si elle
existe.

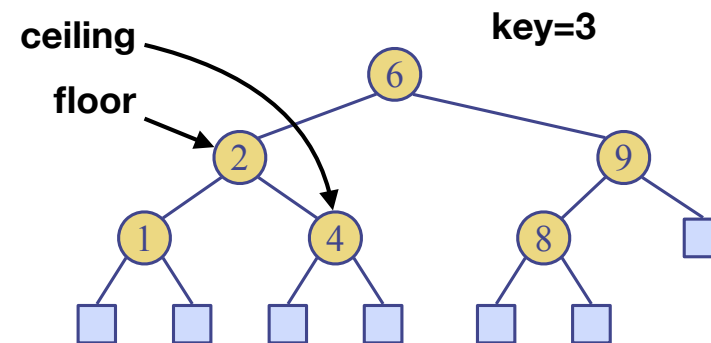
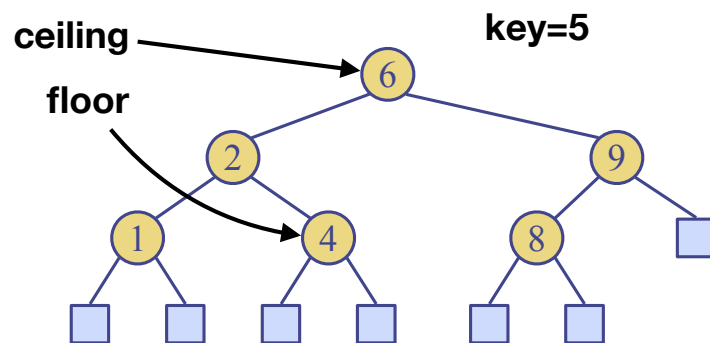
`ceilingEntry` retourne
l'entrée avec la clé la plus
petite plus grande ou égale
à la clé donnée, si elle
existe.

clé dans noeud interne => retourner son Entry



floor : recherche aboutie dans noeud externe => retourner Entry du parent du premier noeud qui est enfant à droite

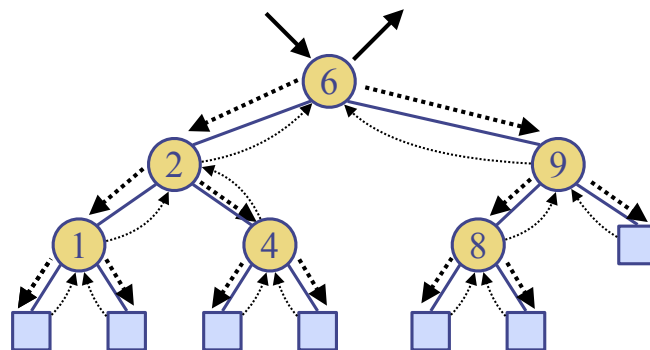
ceiling : recherche aboutie dans noeud externe => retourner Entry du parent du premier noeud qui est enfant à gauche



Voir les codes dans :
 TreeMap.java

Parcours dans l'ordre des clés [inorder]

subMap(fromKey = 1, toKey = 10)

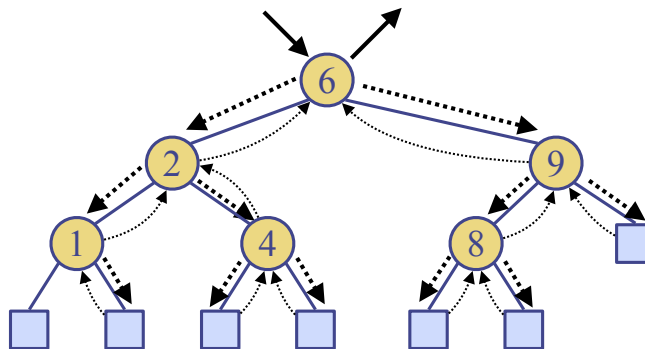


```
p = 6 (root)
p = left( 6 ) = 2
  p = left( 2 ) = 1
    p = left( 1 ) = isExternal
    add( 1 )
    p = right( 1 ) = isExternal
  add( 2 )
  p = right( 2 ) = 4
    p = left( 4 ) = isExternal
    add( 4 )
    p = right( 4 ) = isExternal
add( 6 )
p = right( 6 ) = 9
  p = left( 9 ) = 8
    p = left( 8 ) = isExternal
    add( 8 )
    p = right( 8 ) = isExternal
  add( 9 )
    p = right( 9 ) = isExternal
```

Voir les codes dans :
 TreeMap.java

Parcours dans l'ordre des clés [inorder]

subMap(fromKey = 2, toKey = 10)



```
p = 6 (root)
  p = left( 6 ) = 2
    p = left( 2 ) = 1
      p = right( 1 ) = isExternal
    add( 2 )
      p = right( 2 ) = 4
        p = left( 4 ) = isExternal
      add( 4 )
        p = right( 4 ) = isExternal
    add( 6 )
      p = right( 6 ) = 9
        p = left( 9 ) = 8
          p = left( 8 ) = isExternal
        add( 8 )
          p = right( 8 ) = isExternal
        add( 9 )
          p = right( 9 ) = isExternal
```


Performances des ABR

Considérons une `Map` de n entrées implémentée au moyen d'un arbre binaire de recherche de hauteur h

L'espace utilisé est en $O(n)$

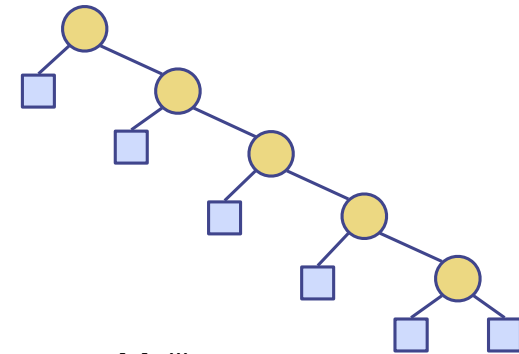
Les méthodes de recherche et de mise à jour sont en $O(h)$

La hauteur h est en $O(n)$ dans le pire cas et $O(\log n)$ dans le meilleur cas

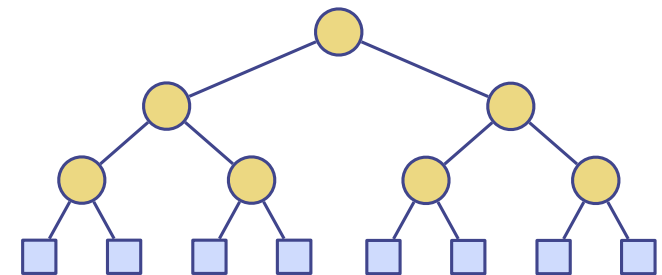
Method	Running Time
size, isEmpty	$O(1)$
get, put, remove	$O(h)$
firstEntry, lastEntry	$O(h)$
ceilingEntry, floorEntry, lowerEntry, higherEntry	$O(h)$
subMap	$O(s + h)$
entrySet, keySet, values	$O(n)$

s est le nombre de résultats.

Pire cas



Meilleur cas



SortedTableMap vs SkipListMap vs TableMap vs ProbeHashMap

Insertion : On insère 5M d'entrées. Pour chaque entrée, on génère une clé (un entier) aléatoire => distribution uniforme.

Recherche : On cherche 5M d'entrées, chacune pour une clé générée aléatoirement.

Suppression : On fait comme pour la recherche mais on supprime l'entrée.

Data structure 5M entrées	Insertion (sec.)	Search (sec.)	Delete (sec.)
Sorted list	$O(n)$	$O(\log n)$	$O(n)$
java.util.ArrayList	417.6	4.2	282.1
Skip list	$O(\log n)$ exp	$O(\log n)$ exp	$O(\log n)$ exp
SkipListMap	9.1	9.4	9.3
java.util.concurrent. ConcurrentSkipListMap	7.8	6.5	7.9
LinkedBinaryTree	$O(n)$	$O(n)$	$O(n)$
TreeMap	6.0	7.1	7.5
Hash table	$O(1)$ exp	$O(1)$ exp	$O(1)$ exp
ProbeHashMap	1.0	0.3	0.3

Pourquoi $O(n)$ pour TreeMap plus rapide que pour ArrayList pour Insertion et Suppression ?

Peut-on faire mieux en garantissant $O(\log n)$ pour toutes les opérations de mise à jour ?

Comment peut-on garantir $O(\log n)$ pour toutes les opérations de mise à jour ?

Équilibrage par rotation (**rotate**) et re-liaison (**relink**)

Le meilleur cas pour un ABR est lorsque l'arbre est équilibré, c-à-d que sa hauteur est $\log n$.

Par conséquent, nous considérons maintenant des techniques d'arbres binaires de recherche équilibrés pour notre implémentation de `TreeMap`

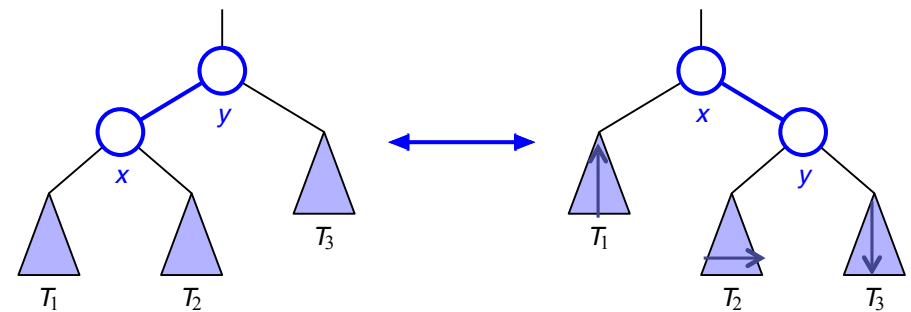
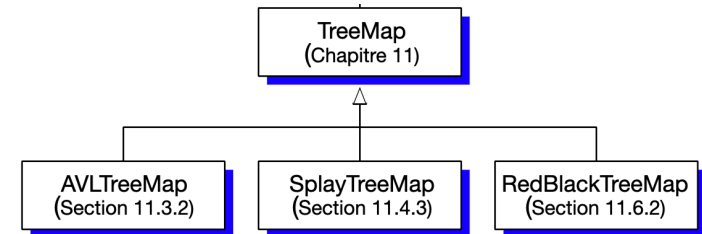
Dans la classe `TreeMap`, nous ajoutons des déclarations triviales de trois méthodes : `rebalanceInsert`, `rebalanceDelete` et `rebalanceAccess`, qui ne font rien :

```
// rebalance methods
protected void rebalanceInsert( Position<Entry<K,V>> p ) {}
protected void rebalanceDelete( Position<Entry<K,V>> p ) {}
protected void rebalanceAccess( Position<Entry<K,V>> p ) {}
```

Les sous-classes de `TreeMap` peuvent surcharger n'importe laquelle de ces méthodes pour implémenter une action pour rééquilibrer un arbre. C'est un autre exemple du **template method design pattern**.

Nous avons aussi implémenter dans `TreeMap` une classe imbriquée d' "arbres équilibrés" :

`BalanceableBinaryTree`, qui sera basée sur une opération de **restructuration** (**restructure**) de trois noeuds.



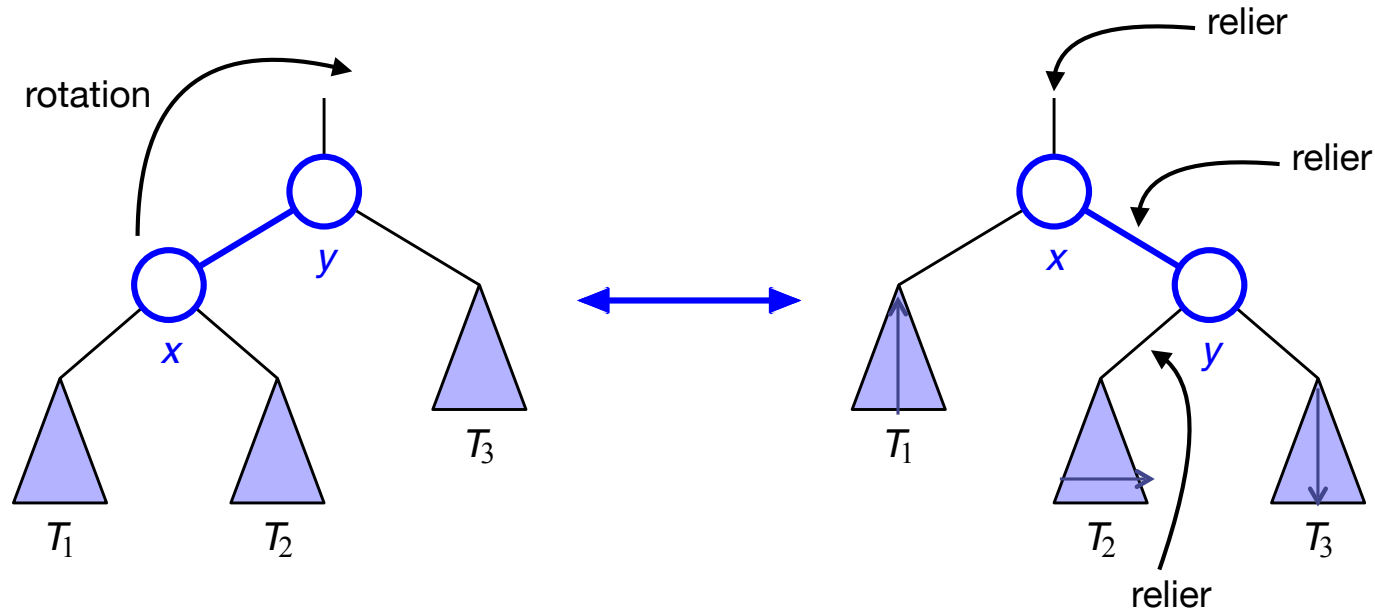
À tout moment pendant les opérations de recherche et de mise à jour, on peut avoir à "rebalancer" l'arbre.

Les structures de données d'arbres binaires de recherche équilibrés (AVL, Splay et **RedBlack**) se distinguent de par la manière dont leurs opérations de rebalancement sont implémentées.

BSTNode<E>

- aux** : De nombreuses stratégies d'équilibrage d'arbre nécessitent une information auxiliaire stockée aux nœuds. Pour alléger le fardeau des sous-classes, on choisit d'ajouter une valeur entière auxiliaire à chaque nœud de la classe `BalanceableBinaryTree`.
- createNode** : On définit une nouvelle classe `BSTNode`, qui hérite elle-même de la classe imbriquée `LinkedBinaryTree.Node`. La nouvelle classe déclare la variable auxiliaire et fournit des méthodes pour obtenir et définir sa valeur.
- Nous attirons l'attention sur une subtilité importante dans notre conception, y compris celle de la sous-classe `LinkedBinaryTree`. Chaque fois qu'une opération de bas niveau sur un arbre nécessite un nouveau nœud, on doit s'assurer que le type de nœud créé est correct. Ici, pour notre arbre équilibré, nous avons besoin de `BSTNode`, qui inclut le champ auxiliaire. Cependant, la création de nœuds se produit au sein d'opérations de bas niveau, telles que `addLeft` et `addRight`, qui résident dans la classe `LinkedBinaryTree`.
- On s'appuie sur la technique **factory method design pattern**. La classe `LinkedBinaryTree` inclut une méthode protégée `createNode` qui est responsable de l'instanciation d'un nouveau nœud du type approprié. Le reste du code de cette classe veille à toujours utiliser la méthode `createNode` lorsqu'un nouveau nœud est nécessaire. On n'a qu'à redéfinir (override) `createNode` dans `BSTNode`, ou tout autre classe de noeuds qui étendrait `Node`.

Pour équilibrer un **ABR**, il faut faire des rotations et reliations



Les opérations pour équilibrer un ABR

- Modifient la structure en $O(1)$
- Maintiennent les propriétés de l'arbre binaire de recherche

Ces trois méthodes sont implémentées dans `BalanceableBinaryTree`

Voir les codes dans :
`TreeMap.java`

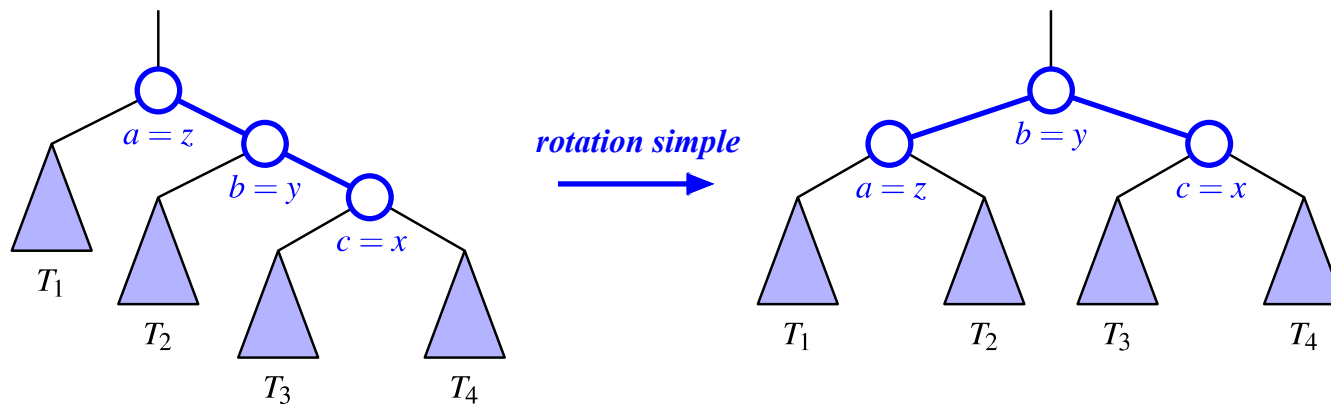
```
// relink a parent node with its oriented child node
private void relink( Node<Entry<K,V>> parent, Node<Entry<K,V>> child, boolean makeLeftChild ) {
    child.setParent( parent );
    if( makeLeftChild )
        parent.setLeft( child );
    else
        parent.setRight( child );
}

// rotate Position p above its parent
public void rotate( Position<Entry<K,V>> p ) {
    Node<Entry<K,V>> x = validate( p );
    Node<Entry<K,V>> y = x.getParent(); // assume it exists
    Node<Entry<K,V>> z = y.getParent(); // grandparent (possibly null)
    if( z == null ) {
        root = x; // x becomes root of the tree
        x.setParent( null );
    } else
        relink( z, x, y == z.getLeft() ); // x becomes direct child of z
    // now rotate x and y, including transfer of middle tree
    if( x == y.getLeft() ) {
        relink( y, x.getRight(), true ); // x's right child becomes y's left
        relink( x, y, false );          // y becomes x's right child
    }
    else {
        relink( y, x.getLeft(), false ); // x's left child becomes y's right
        relink( x, y, true );            // y becomes x's left child
    }
}

// trinode restructuring of Position x with its parent/grandparent
public Position<Entry<K,V>> restructure( Position<Entry<K,V>> x ) {
    Position<Entry<K,V>> y = parent( x );
    Position<Entry<K,V>> z = parent( y );
    if( ( x == right( y ) ) == ( y == right( z ) ) ) { // matching alignment
        rotate( y ); // single rotation (of y)
        return y;    // y is new subtree root
    } else {        // opposite alignments (zig-zag)
        rotate( x ); // double rotation (of x)
        rotate( x );
        return x;    // x is new subtree root
    }
}

} // ----- End of nested BalanceableBinaryTree class -----
```

Restructuration de trois noeuds (rotation simple)



Nous considérons une position x , son parent y et son grand-parent z .
L'objectif est de restructurer le sous-arbre pour réduire la longueur du chemin jusqu'à x et ses sous-arbres.

Restructuration de trois noeuds (rotation simple) (cas vers la droite)

restructure(**x**) :

entrée : position **x** et ABR **T** (👉 parent de **x** est **y** ; grand-parent **z**)

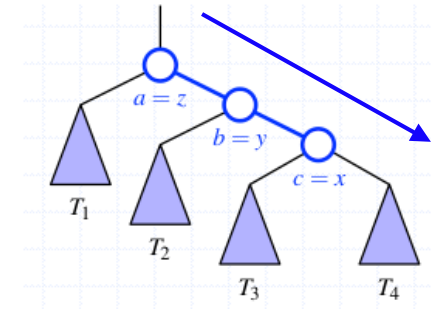
sortie : **T** après la restructuration des trois noeuds

opération : rotation simple autour de **y** (**rotate**) en 3 (re)liasons (**relink**)

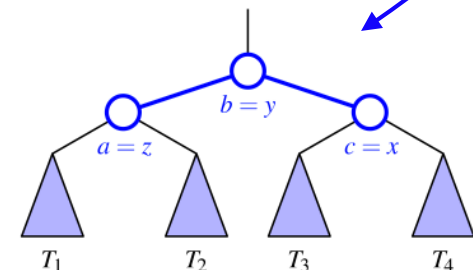
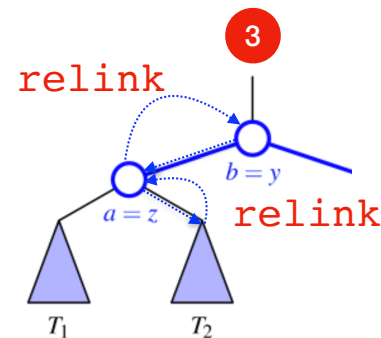
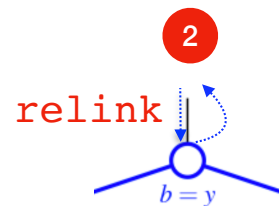
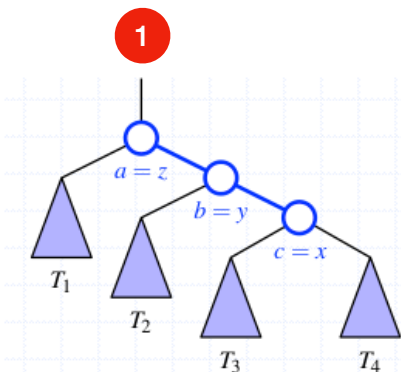
- 1 On définit (a, b, c) comme étant les noeuds x, y et z dans l'ordre de gauche à droite (inorder) et (T1, T2, T3, T4) les 4 sous-arbres de x, y et z
- 2 On met b comme racine du sous-arbre gauche ou droit selon le cas (**relink**)
- 3 On met a comme enfant gauche de b (**relink**) et T2 comme sous-arbre droit de a (**relink**)

Ce qui n'a pas bougé :

c est l'enfant droit de b, T3 et T4 les sous-arbres gauche et droit de c, et T1 le sous-arbre gauche de a.



rotate(y)



Restructuration de trois noeuds (rotation simple) (cas symétrique vers la gauche)

restructure(**x**) :

entrée : position **x** et ABR **T** (👉 parent de **x** est **y** ; grand-parent **z**)

sortie : **T** après la restructuration des trois noeuds

opération : rotation simple autour de **y** (**rotate**) en 3 (re)liions (**relink**)

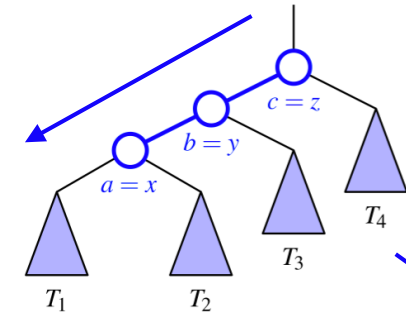
1 On définit (a, b, c) comme étant les noeuds x, y et z dans l'ordre de gauche à droite (inorder) et (T_1, T_2, T_3, T_4) les 4 sous-arbres de x, y et z

2 On met b comme racine du sous-arbre, gauche ou droit selon le cas (**relink**)

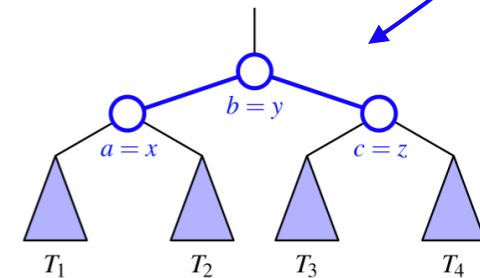
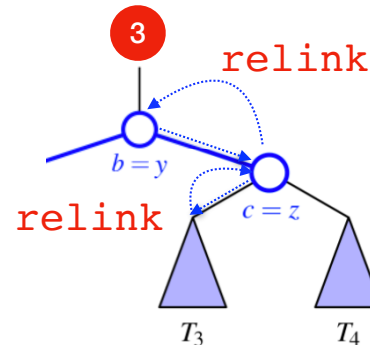
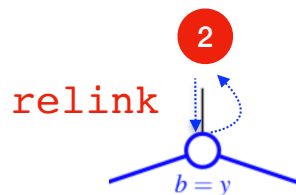
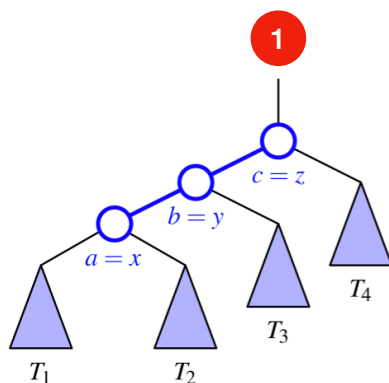
3 On met c comme enfant droit de b (**relink**) et T_3 comme sous-arbre gauche de c (**relink**)

Ce qui n'a pas bougé :

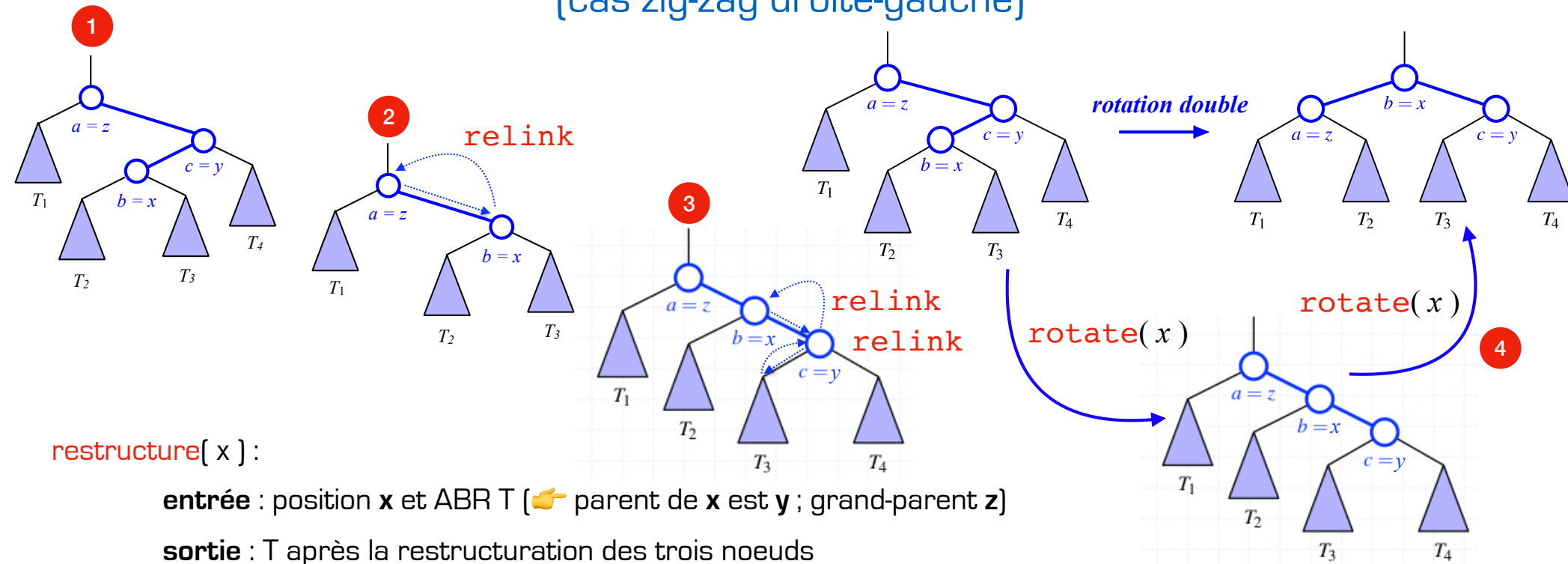
a est l'enfant gauche de b , T_1 et T_2 les sous-arbres gauche et droit de a et T_4 le sous-arbre droit de c .



rotate(y)



Restructuration de trois noeuds (rotation double) (cas zig-zag droite-gauche)



restructure[x] :

entrée : position x et ABR T (👉 parent de x est y ; grand-parent z)

sortie : T après la restructuration des trois noeuds

opération : rotation simple autour de x (**rotate**) en 3 (re)liations (**relink**)

1 On définit $[a, b, c]$ comme étant les noeuds x, y et z dans l'ordre de gauche à droite (inorder) et $[T_1, T_2, T_3, T_4]$ les 4 sous-arbres de x, y et z

2 On met b comme enfant droit de z (**relink**)

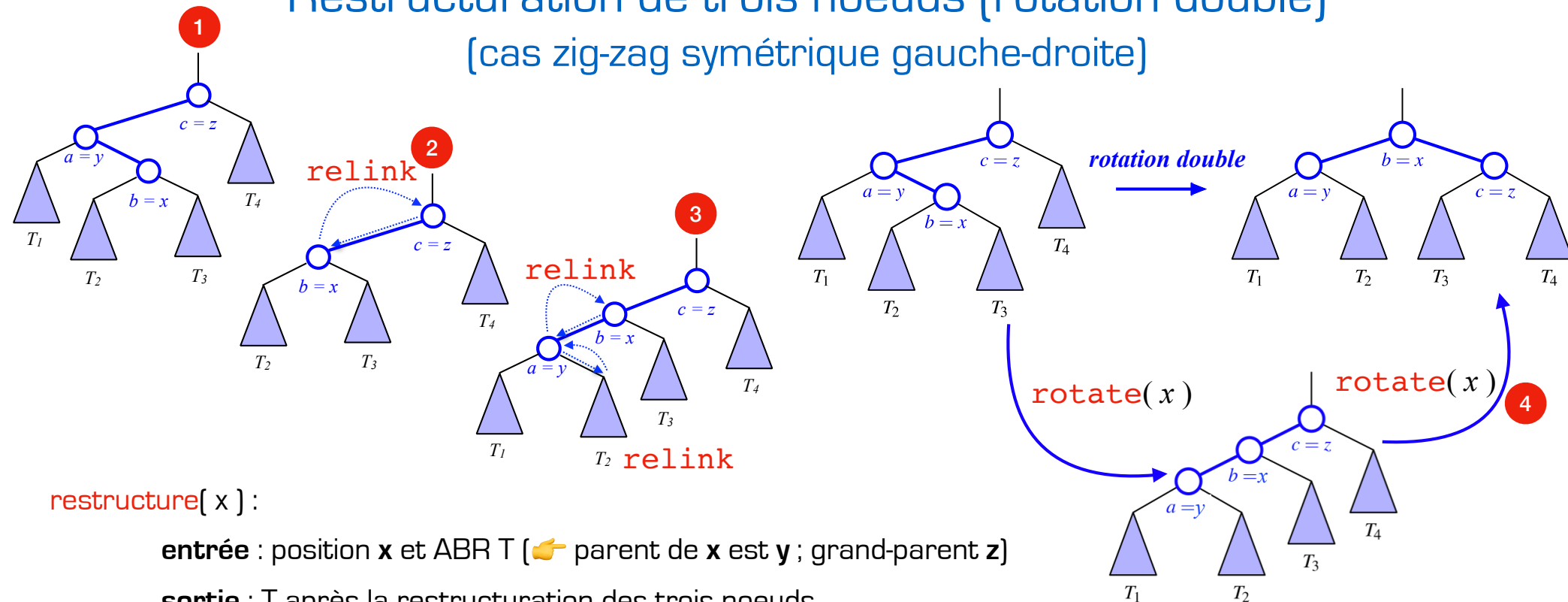
3 On met c comme enfant droit de b (**relink**) et T_3 comme sous-arbre gauche de c (**relink**)

Ce qui n'a pas bougé :

T_1 est le sous-arbre gauche de a , T_2 est le sous-arbre gauche de b et T_4 est le sous-arbre droit de c .

4 On se retrouve dans un cas de rotation simple, donc on applique une 2ème rotation autour de x (**rotate**)

Restructuration de trois noeuds (rotation double) [cas zig-zag symétrique gauche-droite]



restructure(x) :

entrée : position x et ABR T (👉 parent de x est y ; grand-parent z)

sortie : T après la restructuration des trois noeuds

opération : rotation simple autour de x (**rotate**) en 3 (re)liaisons (**relink**)

1 On définit (a, b, c) comme étant les noeuds x, y et z dans l'ordre de gauche à droite (inorder) et (T_1, T_2, T_3, T_4) les 4 sous-arbres de x, y et z

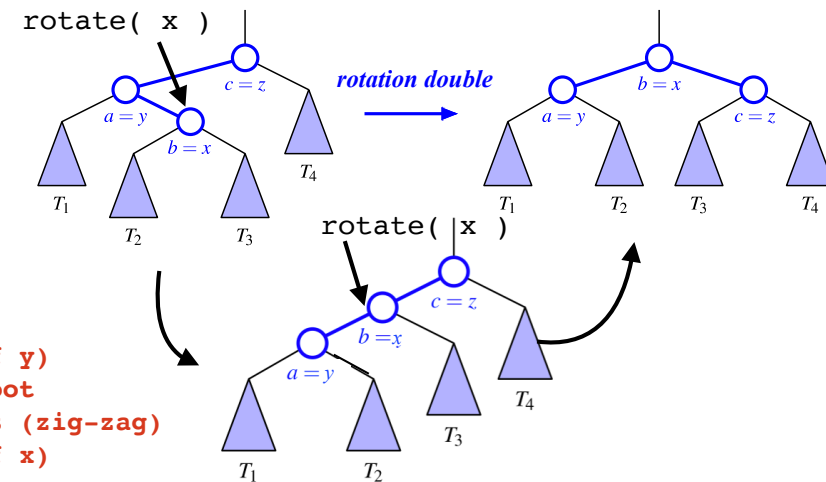
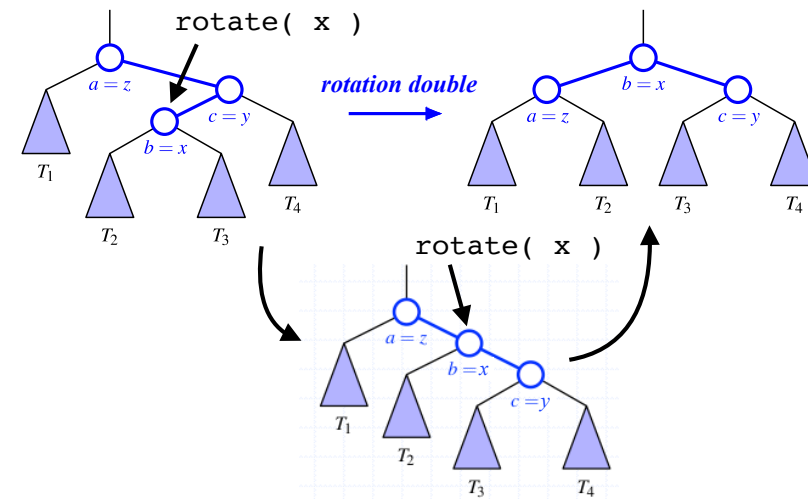
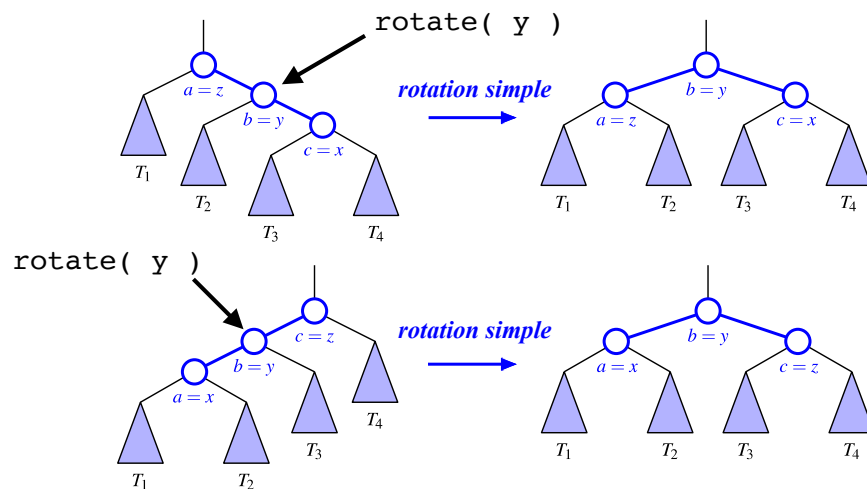
2 On met b comme enfant gauche de z (**relink**)

3 On met a comme enfant gauche de b (**relink**) et T_2 comme sous-arbre droit de a (**relink**)

Ce qui n'a pas bougé :

T_1 est le sous-arbre gauche de a , T_3 est le sous-arbre droit de b et T_4 est le sous-arbre droit de c .

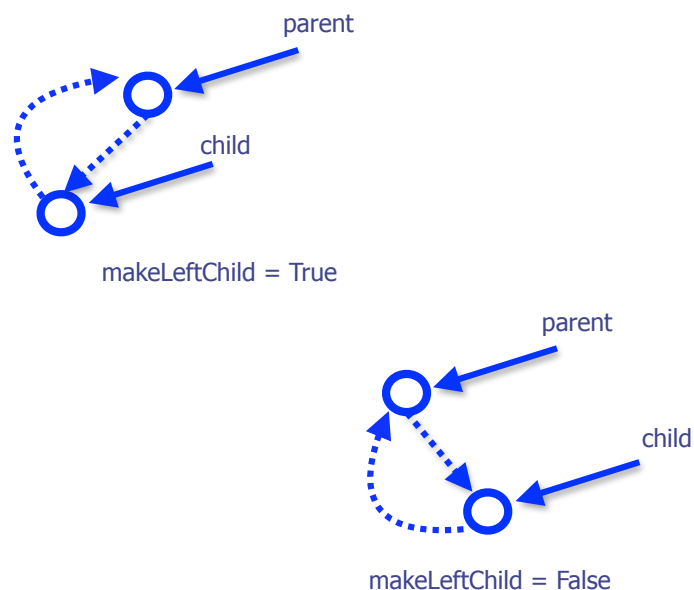
4 On se retrouve dans un cas de rotation simple, donc on applique une 2ème rotation autour de x (**rotate**)



```
// trinode restructuring of Position x with its parent/grandparent
public Position<Entry<K,V>> restructure( Position<Entry<K,V>> x ) {
    Position<Entry<K,V>> y = parent( x );
    Position<Entry<K,V>> z = parent( y );
    if( ( x == right( y ) ) == ( y == right( z ) ) ) { // matching alignment
        rotate( y );
        return y;
    } else {
        rotate( x );
        rotate( x );
        return x;
    }
}
```

// matching alignment
 // single rotation (of y)
 // y is new subtree root
 // opposite alignments (zig-zag)
 // double rotation (of x)
 // x is new subtree root

Relier un enfant à gauche ou à droite d'un parent [relink]

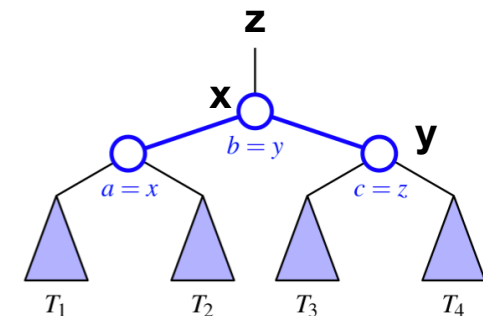
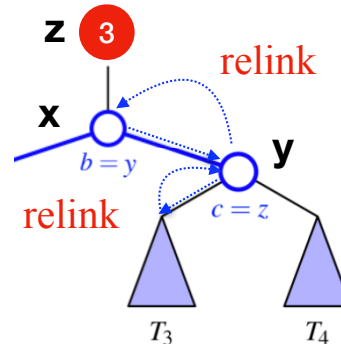
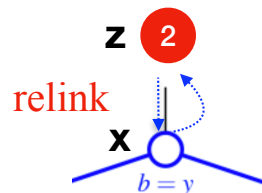
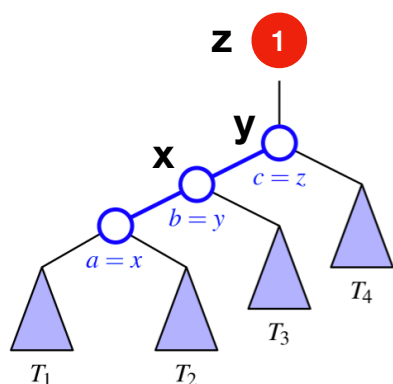
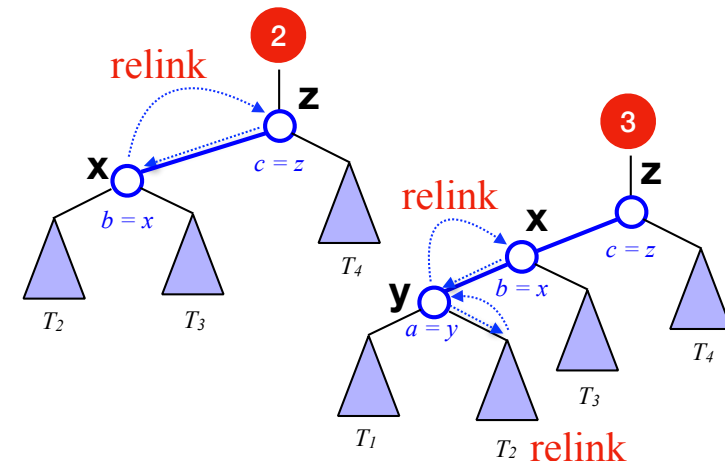
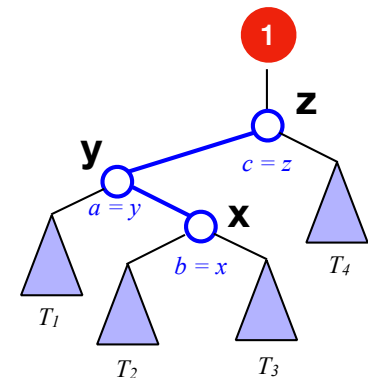


```
// relink a parent node with its oriented child node
private void relink( Node<Entry<K,V>> parent, Node<Entry<K,V>> child, boolean makeLeftChild ) {
    child.setParent( parent );
    if( makeLeftChild )
        parent.setLeft( child );
    else
        parent.setRight( child );
}
```

Rotation simple

[rotate]

```
// rotate Position p above its parent
public void rotate( Position<Entry<K,V>> p ) {
    Node<Entry<K,V>> x = validate( p );
    Node<Entry<K,V>> y = x.getParent(); // assume it exists
    Node<Entry<K,V>> z = y.getParent(); // grandparent (possibly null)
    if( z == null ) {
        root = x; // x becomes root of the tree
        x.setParent( null );
    } else {
        relink( z, x, y == z.getLeft() ); // x becomes direct child of z
        // now rotate x and y, including transfer of middle tree
        if( x == y.getLeft() ) {
            relink( y, x.getRight(), true ); // x's right child becomes y's left
            relink( x, y, false );           // y becomes x's right child
        } else {
            relink( y, x.getLeft(), false ); // x's left child becomes y's right
            relink( x, y, true );             // y becomes x's left child
        }
    }
}
```



👉 Attention : `x = validate( p )` mais la rotation se fait sur `b = y`

